

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

OFFLINE CLASS RULES

TO BE FOLLOWED VERY STRICTLY

- 1. BE PRESENT IN THE CLASS ATLEAST 5 MINUTES BEFORE THE START OF THE CLASSES**
- 2. STUDENTS LATE BY MORE THAN 5 MINUTES WILL NOT BE ALLOWED TO ENTER THE CLASS**
- 3. USE OF CELLPHONES & LAPTOPS STRICTLY PROHIBITED INSIDE THE CLASS**
- 4. CELL PHONES & LAPTOPS SHOULD NOT BE KEPT ON THE DESK**
- 5. CELLPHONES SHOULD BE KEPT EITHER IN YOUR POCKET OR BACKPACK/BAG YOU CARRY**
- 6. SLEEPING IN CLASS IS TOTALLY PROHIBITED**
- 7. BRING A NOTE BOOK FOR THIS COURSE AND MAKE NOTES IN THIS BOOK DURING THE CLASS**
- 8. NO DRINKING & NO EATING IS ALLOWED IN CLASS (PLEASE NOTE THAT CLASSROOM IS NOT CANTEEN)**

SOC 2040

SYSTEM PROGRAMMING



WEEK 4 LECTURE 2

INTEGER ARITHMETIC

Unsigned Addition

Operands: w bits



+ v



True Sum: $w+1$ bits

$u + v$



Discard Carry: w bits

$\text{UAdd}_w(u, v)$



- ★ Standard Addition Function

- Ignores carry output

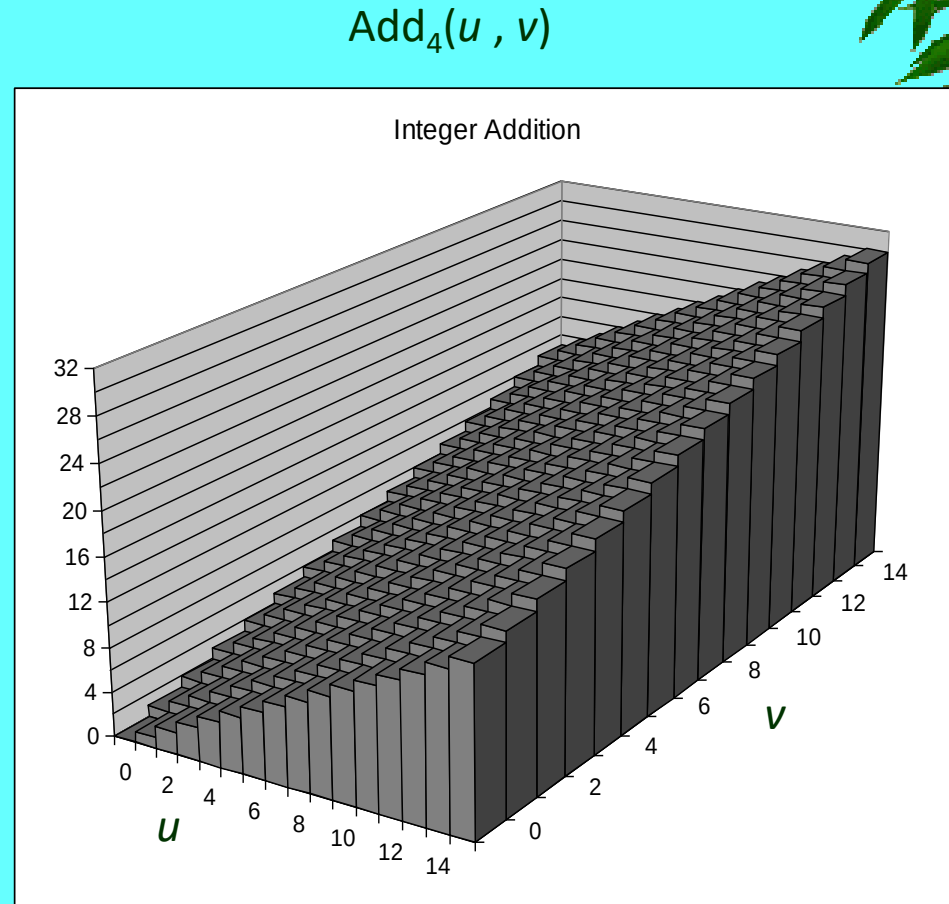
- ★ Implements Modular Arithmetic

$$s = \text{UAdd}_w(u, v) = u + v \bmod 2^w$$

Visualizing (Mathematical) Integer Addition

★ Integer Addition

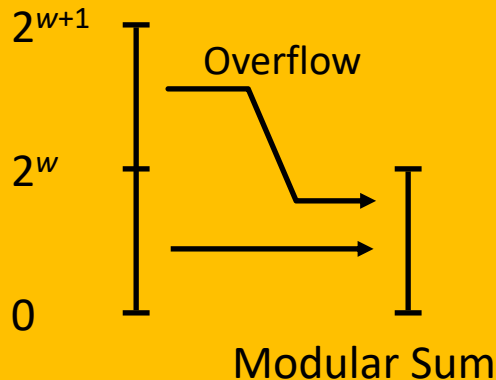
- 4-bit integers u , v
- Compute true sum $\text{Add}_4(u, v)$
- Values increase linearly with u and v
- Forms planar surface



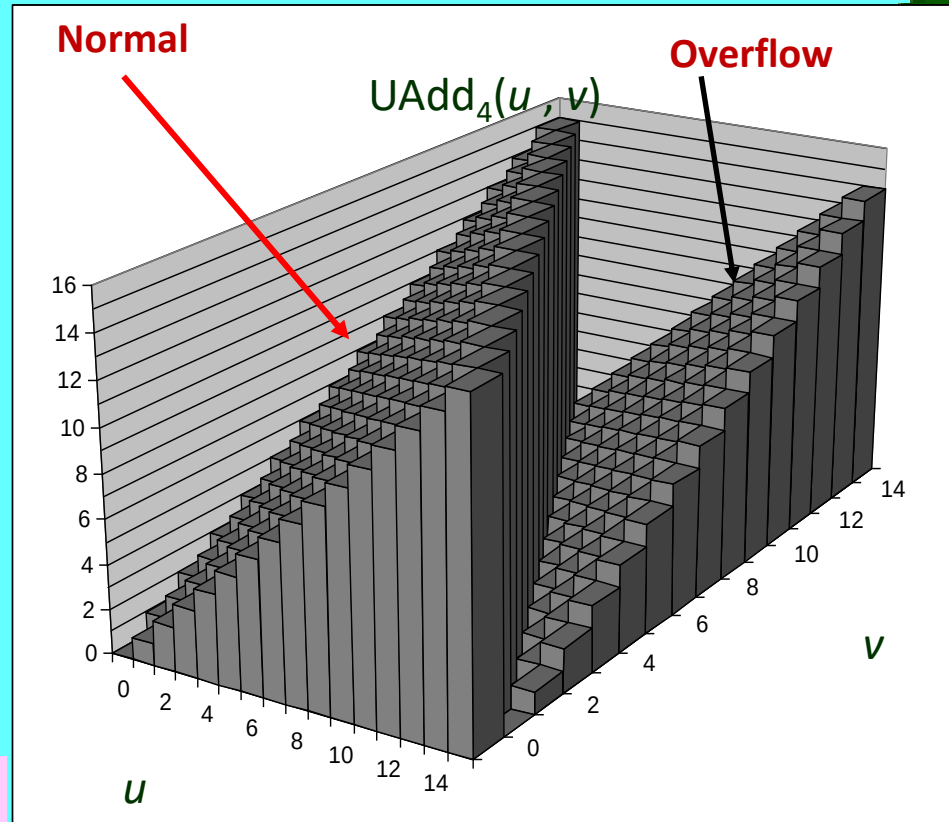
Visualizing Unsigned Addition

- ★ Unsigned arithmetic can be viewed as a form of modular arithmetic. Unsigned addition is equivalent to computing the sum modulo 2^w

- ★ Wraps Around
 - If true sum $\geq 2^w$
 - At most once
- True Sum



$$x +_w^u y = \begin{cases} x + y, & x + y < 2^w \\ x + y - 2^w, & 2^w \leq x + y < 2^{w+1} \end{cases} \quad (2.11)$$



$x=9=1001$, $y=12=1100$
 $x+y = 1001 + 1100 = 10101=21$
Since $w=4$, $x+y = 0101 = 5$
i.e., $21 \bmod 16 = 5$

Two's Complement Addition

Given x, y in the range $-2^{w-1} \leq x, y \leq 2^{w-1} - 1$, their sum is in the range $-2^w \leq x+y \leq 2^w - 2$ requiring $w+1$ bits

Operands: w bits



True Sum: $w+1$ bits



Discard Carry: w bits



★ TAdd and UAdd have Identical Bit-Level Behavior

– Signed vs. unsigned addition in C:

```
int s, t, u, v;
```

```
s = (int) ((unsigned) u + (unsigned) v);
```

```
t = u + v
```

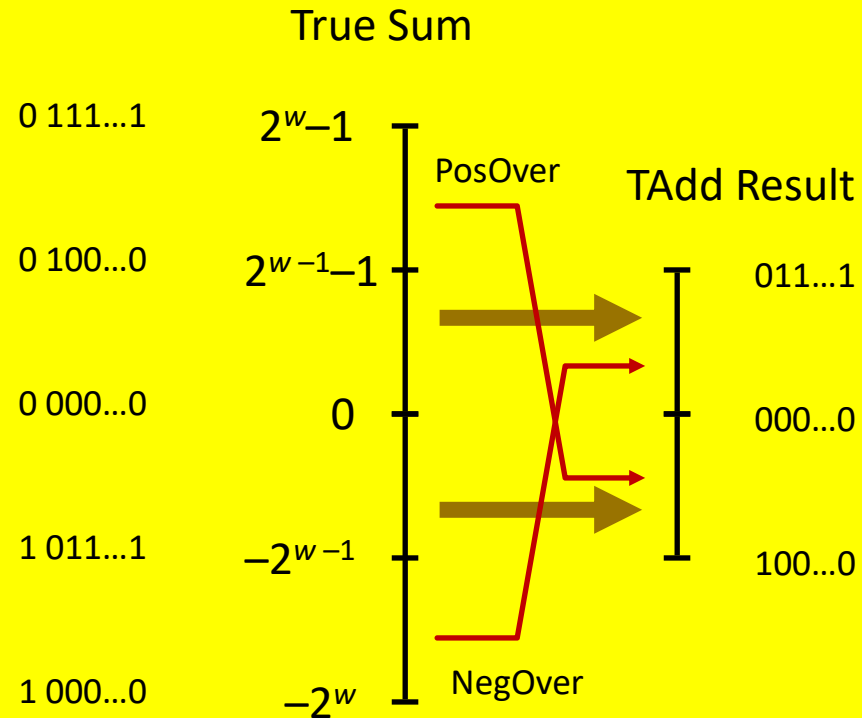
– Will give $s == t$

$$x +_w^t y \doteq U2T_w(T2U_w(x) +_w^u T2U_w(y)) \quad (2.13)$$

TWO'S COMPLEMENT ADDITION

★ Functionality

- True sum requires $w+1$ bits
- Drop off MSB
- Treat remaining bits as 2's comp. integer



$$x + y = U2T_w(T2U_w(x) + T2U_w(y))$$

$$= U2T_w [(x_{w-1} 2^w + x + y_{w-1} 2^w + y) \bmod 2^w]$$

$$= U2T_w [(x + y) \bmod 2^w]$$

The terms $x_{w-1} 2^w$ and $y_{w-1} 2^w$ drop out since they equal 0 modulo 2^w .

OVERFLOW -Two's Complement Addition

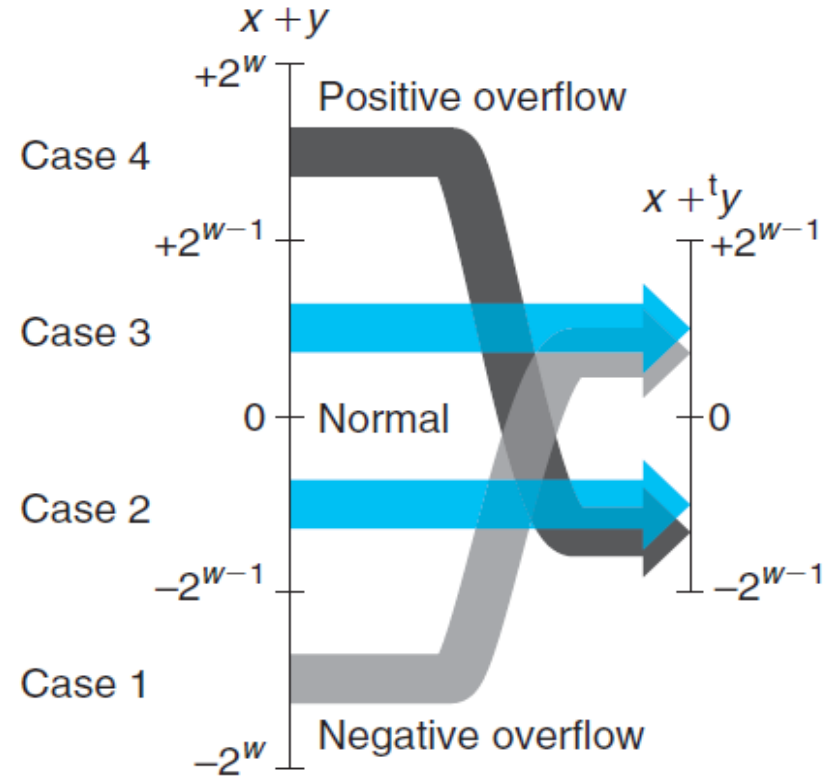
Compute $z = x + y$

* **Case 4** : When $x + y$ is greater than $2^{w-1}+1$, then there is Positive Overflow (Overflow)

* **Case 3** : $0 \leq z < 2^{w-1}$

* **Case 2** : $-2^{w-1} \leq z < 0$

* **Case 1** : When $x + y$ is less than -2^{w-1} , then there is Negative Overflow (Underflow)



$$x + {}^t y = \begin{cases} x + y - 2^w, & 2^{w-1} \leq x + y \quad \text{Positive overflow} \\ x + y, & -2^{w-1} \leq x + y < 2^{w-1} \quad \text{Normal} \\ x + y + 2^w, & x + y < -2^{w-1} \quad \text{Negative overflow} \end{cases} \quad (2.14)$$

Visualizing 2's Complement Addition

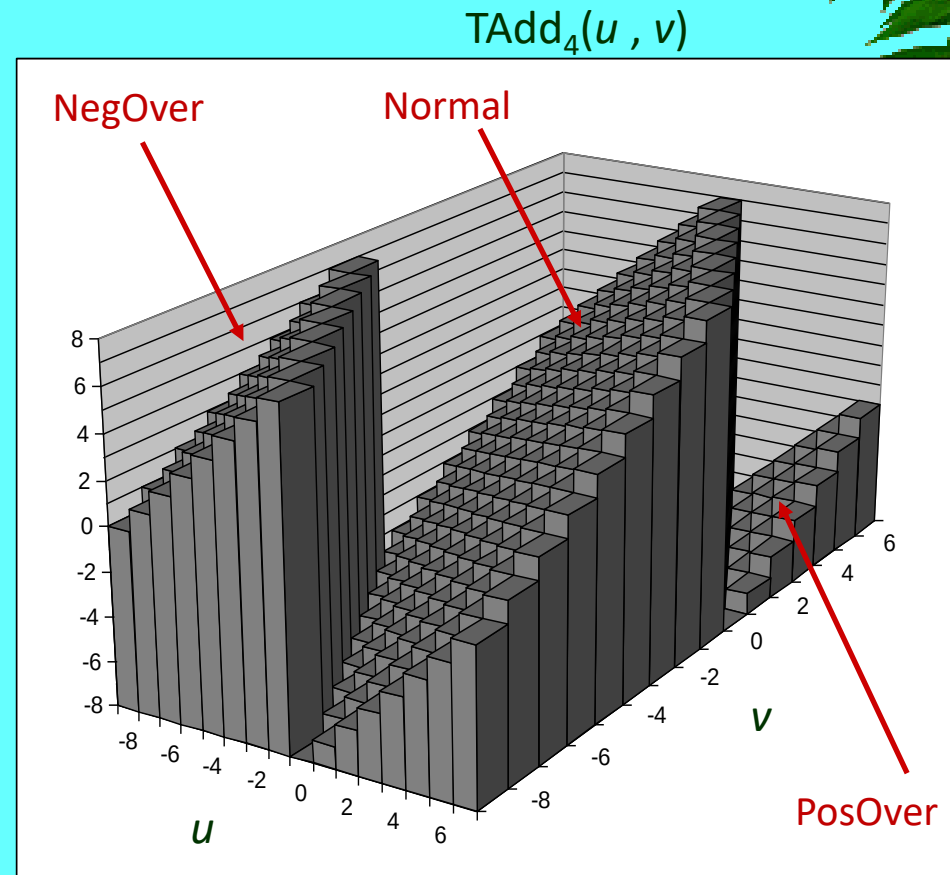
- ★ Values

- 4-bit two's comp.
- Range from -8 to +7

- ★ Wraps Around

- If $\text{sum} \geq 2^{w-1}$
 - ★ Becomes negative
 - ★ At most once
- If $\text{sum} < -2^{w-1}$
 - ★ Becomes positive
 - ★ At most once

$$\begin{aligned}x +_w^t y &= U2T_w(T2U_w(x) +_w^u T2U_w(y)) \\&= U2T_w[(x_{w-1}2^w + x + y_{w-1}2^w + y) \bmod 2^w] \\&= U2T_w[(x + y) \bmod 2^w]\end{aligned}$$



Two's Complement Addition & Subtraction

Examples : ADDITION $t = a + b$

a=+127 01111111 +
b=+127 01111111
t = 11111110

a=-128 10000000+
b=-127 10000001
00000001

NEGATIVE RESULT - OVERFLOW

POSITIVE RESULT - OVERFLOW

Conditions for detecting overflow in Two's Complement Addition

If $a > 0$ & $b > 0$ and $t < 0$ then there is an OVERFLOW

If $a < 0$ & $b < 0$ and $t > 0$ then there is an OVERFLOW

Examples : SUBTRACTION $t = a - b$

a=+127 0111 1111 -
b=-127 1000 0001
11111110

a=-128 10000000 -
b=+127 01111111
00000001

NEGATIVE RESULT - OVERFLOW

POSITIVE RESULT - OVERFLOW

Conditions for detecting overflow in Two's Complement Subtraction

If $a > 0$ & $b < 0$ and $t < 0$ then there is an OVERFLOW

If $a < 0$ & $b > 0$ and $t > 0$ then there is an OVERFLOW

Two's Complement Negation

- * We can see that every number x in the range $-2^{w-1} \leq x < 2^{w-1}$ has an additive inverse under $+^t_w$ as follows.
- * First, for $x \neq -2^{w-1}$, we can see that its additive inverse is simply $-x$.
- * That is, we have $-2^{w-1} < -x < 2^{w-1}$ and $-x +^t_w x = -x + x = 0$.
- * For $x = -2^{w-1} = \text{Tmin}_w$, on the other hand, $-x = 2^{w-1}$ cannot be represented as a w bit number.
- * We claim that this special value has itself as the additive inverse under $+^t_w$.
- * The value of $-2^{w-1} +^t_w -2^{w-1}$ is given by the third case of Equation 2.14, since $-2^{w-1} + -2^{w-1} = -2^w$.
- * This gives $-2^{w-1} +^t_w -2^{w-1} = -2^w + 2^w = 0$.
- * From this analysis, we can define the two's-complement negation operation $+^t_w$ for x in the range $-2^{w-1} \leq x < 2^{w-1}$ as

$$-^t_w x = \begin{cases} -2^{w-1}, & x = -2^{w-1} \\ -x, & x > -2^{w-1} \end{cases} \quad (2.15)$$

If $x = +8 = 00001000$, $\text{NEG}(x) = 0 - 8 = -8 = 2\text{'s complement}(8)$
 $00000000 - 00001000 = 11111000 = -8$

Negation: Complement & Increment

- ★ Claim: Following Holds for 2's Complement

$$\sim x + 1 == -x$$

- ★ Complement

– Observation: $\sim x + x == 1111\dots111 == -1$

x	1	0	0	1	1	1	0	1	
+	~x	0	1	1	0	0	0	1	0
-1	1	1	1	1	1	1	1	1	

Complement & Increment Examples

x = 15213

	Decimal	Hex	Binary
x	15213	3B 6D	00111011 01101101
~x	-15214	C4 92	11000100 10010010
~x+1	-15213	C4 93	11000100 10010011
y	-15213	C4 93	11000100 10010011

x = 0

	Decimal	Hex	Binary
0	0	00 00	00000000 00000000
~0	-1	FF FF	11111111 11111111
~0+1	0	00 00	00000000 00000000

Multiplication

- * Goal: **Computing Product of w -bit numbers x, y**
 - Either signed or unsigned
- * But, exact results can be bigger than w bits
 - Unsigned: up to $2w$ bits
 - * Result range: $0 \leq x * y \leq (2^w - 1)^2 = 2^{2w} - 2^{w+1} + 1$
 - Two's complement min (negative): Up to $2w-1$ bits
 - * Result range: $x * y \geq (-2^{w-1}) * (2^{w-1} - 1) = -2^{2w-2} + 2^{w-1}$
 - Two's complement max (positive): Up to $2w$ bits, but only for $(TMin_w)^2$
 - * Result range: $x * y \leq (-2^{w-1})^2 = 2^{2w-2}$
- * So, maintaining exact results...
 - would need to keep expanding word size with each product computed
 - is done in software, if needed
 - * e.g., by “arbitrary precision” arithmetic packages

Unsigned Multiplication in C

$$x *^u_w y = (x \cdot y) \bmod 2^w \quad (2.16)$$

Operands: w bits

u 

$*$ v 

True Product: $2 \cdot w$ bits

$u \cdot v$ 

Discard w bits: w bits

$\text{UMult}_w(u, v)$ 

- ★ Standard Multiplication Function

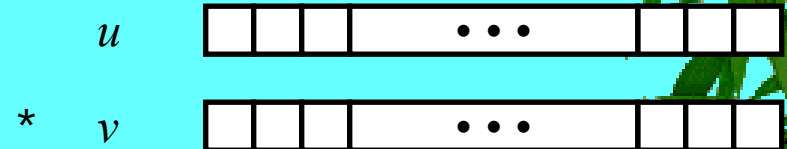
- Ignores high order w bits

- ★ Implements Modular Arithmetic

$$\text{UMult}_w(u, v) = u \cdot v \bmod 2^w$$

Signed Multiplication in C

Operands: w bits



True Product: $2w$ bits



Discard w bits: w bits



$$x *_w^t y = U2T_w((x \cdot y) \bmod 2^w) \quad (2.17)$$

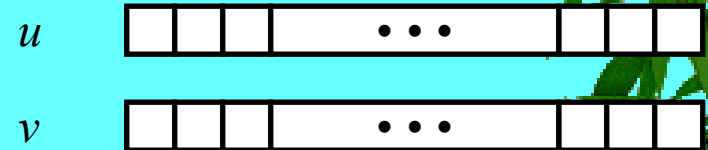
★ Standard Multiplication Function

- Ignores high order w bits
- Some of which are different for signed vs. unsigned multiplication
- Lower bits are the same

Signed Multiplication in C

+	*	+	=	+
-	*	-	=	+
+	*	-	=	-
-	*	+	=	-

Operands: w bits



True Product: $2*w$ bits



Discard w bits: w bits



$$x *_w^t y = U2T_w((x \cdot y) \bmod 2^w) \quad (2.17)$$

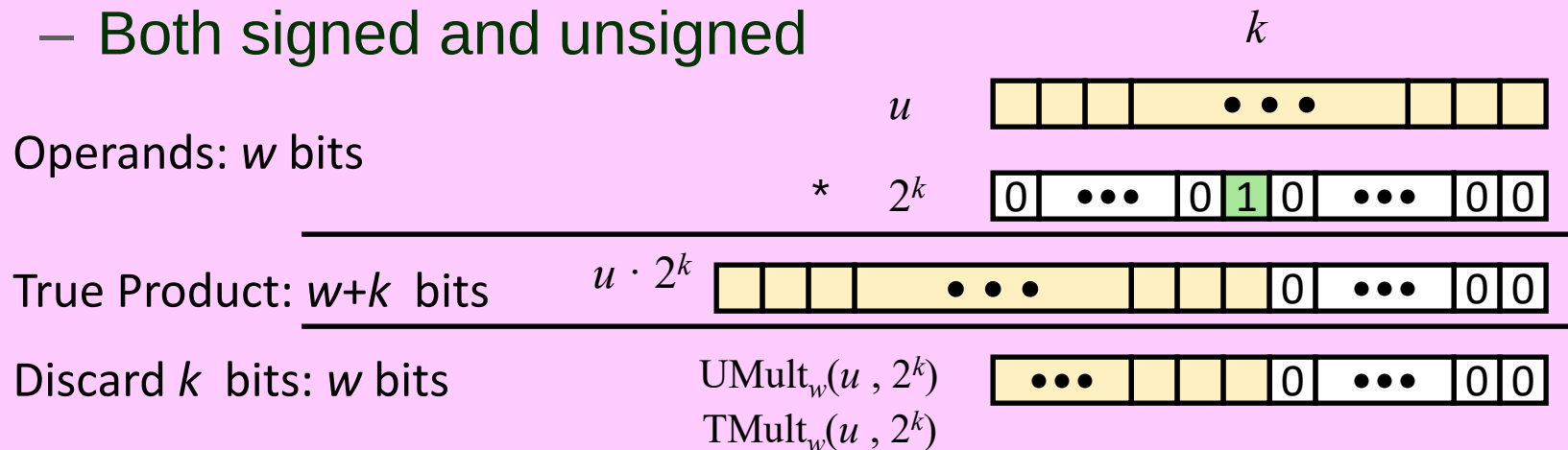
- ★ Multiplying Two negative numbers x' & y' is same as multiplying two positive numbers x & y

$$\begin{aligned}
 (x' \cdot y') \bmod 2^w &= [(x + x_{w-1}2^w) \cdot (y + y_{w-1}2^w)] \bmod 2^w & (2.18) \\
 &= [x \cdot y + (x_{w-1}y + y_{w-1}x)2^w + x_{w-1}y_{w-1}2^{2w}] \bmod 2^w \\
 &= (x \cdot y) \bmod 2^w
 \end{aligned}$$

Multiplying Constants

★ Power-of-2 Multiply with Shift Operation

- $u \ll k$ gives $u * 2^k$
- Both signed and unsigned



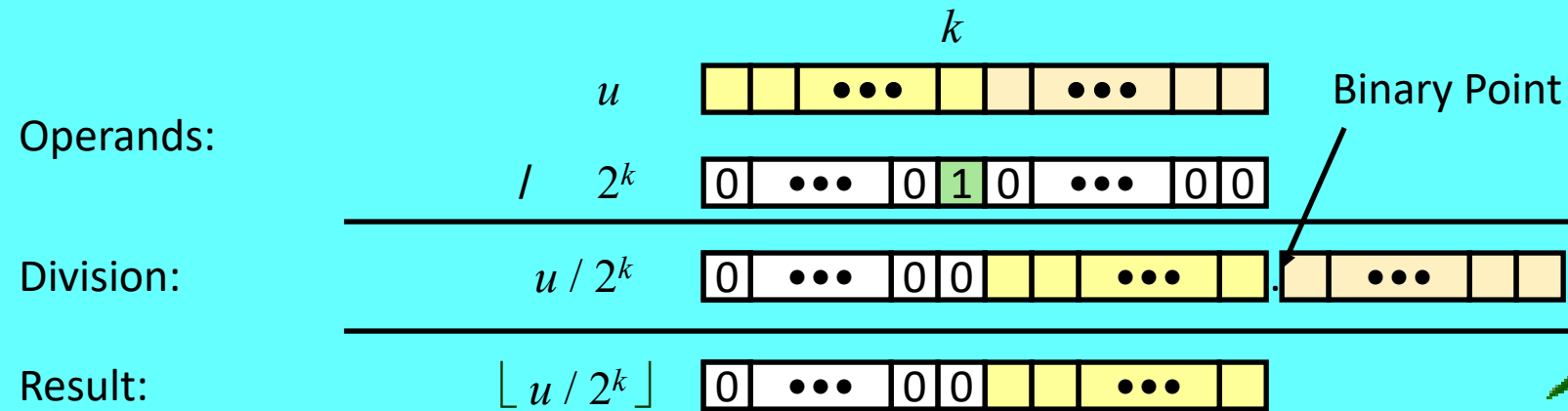
★ Examples

- $u \ll 3 \quad == \quad u * 2^3 \quad = \quad u * 8$
- $(u \ll 5) - (u \ll 3) == u * 2^5 - u * 2^3 = u * 24$
- Most machines shift and add faster than multiply
 - ★ Compiler generates this code automatically

Unsigned Power-of-2 Divide with Shift

★ Quotient of Unsigned by Power of 2

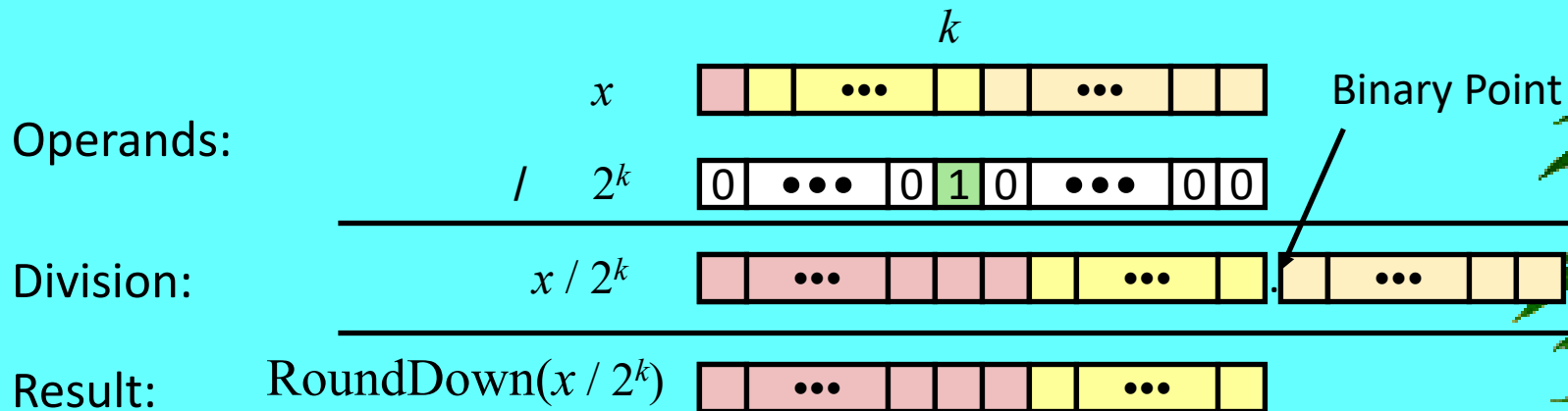
- $u \gg k$ gives $\lfloor u / 2^k \rfloor$
- Uses logical shift



	Division	Computed	Hex	Binary
x	15213	15213	3B 6D	00111011 01101101
x >> 1	7606.5	7606	1D B6	00011101 10110110
x >> 4	950.8125	950	03 B6	00000011 10110110
x >> 8	59.4257813	59	00 3B	00000000 00111011

Signed Power-of-2 Divide with Shift

- ★ Quotient of Signed by Power of 2
 - $x \gg k$ gives $\lfloor x / 2^k \rfloor$
 - Uses arithmetic shift
 - Rounds wrong direction when $u < 0$



	Division	Computed	Hex	Binary
y	-15213	-15213	C4 93	11000100 10010011
y >> 1	-7606.5	-7607	E2 49	11100010 01001001
y >> 4	-950.8125	-951	FC 49	11111100 01001001
y >> 8	-59.4257813	-60	FF C4	11111111 11000100

Rewrite the following expressions involving x & y variables by replacing the multiplication (*) and division (/) operations with the simple operations left shift (<<), right shift (>>), addition(+) and subtraction(-) operations :

1. $x * 29 = x * 32 - x * 2 - x = x * 2^5 - x * 2^1 - x$
 $= x << 5 - x << 1 - x$
2. $y * 144 = y * 128 + y * 16 = y * 2^7 + y * 2^4$
 $= y << 7 + y << 4$
3. $x * 195 = x * 128 + x * 64 + x * 2 + x = x * 2^7 + x * 2^6 + x * 2^1 + x$
 $= x << 7 + x << 6 + x << 1 + x$
4. $x * 246 = x * 256 - x * 8 - x * 2 = x * 2^8 - x * 2^3 - x * 2^1$
 $= x << 8 - x << 3 - x << 1$
5. $y * 1000 = y * 1024 - y * 16 - y * 8 = y * 2^{10} - y * 2^4 - y * 2^3$
 $= y << 10 - y << 4 - y << 3$
6. $y * 2000 = y * 2048 - y * 32 - y * 16 = y * 2^{11} - y * 2^5 - y * 2^4$
 $= y << 11 - y * 5 - y * 4$
7. $y * 4000 = y * 4096 - y * 64 - y * 32 = y * 2^{12} - y * 2^6 - y * 2^5$
 $= y << 12 - y << 6 - y << 5$
8. $x * 65567 = x * 65536 + x * 32 - x = x * 2^{16} + x * 2^5 - x$
 $= x << 16 + x << 5 - x$
9. $x / 8 = x / 2^3 = x >> 3$
10. $x / 64 = x / 2^6 = x >> 6$
11. $x / 2048 = x / 2^{11} = x >> 11$
12. $x / 4096 = x / 2^{12} = x >> 12$

Rewrite the following expressions involving x variables using only the multiplication (*) or division (/) operation.

1. $x \ll 10 - x \ll 4 - x \ll 3 = x * 2^{10} - x * 2^4 - x * 2^3$
 $= x * 1024 - x * 16 - x * 8 = x * 1000$

2. $x \ll 6 - x \ll 2 + x = x * 2^6 - x * 2^2 + x$
 $= x * 64 - x * 4 + x = x * 61$

3. $x \ll 7 + x \ll 3 + x \ll 1 = x * 2^7 + x * 2^3 + x * 2^1$
 $= x * 128 + x * 8 + x * 2 = x * 138$

4. $x \ll 12 + x \ll 2 - x \ll 4 - x = x * 2^{12} + x * 2^2 - x * 2^4 - x$
 $= x * 4096 + x * 4 - x * 16 - x = 4983$

5. $x \ll 13 - x \ll 7 - x \ll 6 = x * 2^{13} - x * 2^7 - x * 2^6$
 $= x * 8192 - x * 128 - x * 64 = x * 8000$

6. $x \ll 15 + x \ll 12 + x \ll 11 + x \ll 10 + x \ll 6 - x$
 $= x * 2^{15} + x * 2^{12} + x * 2^{11} + x * 2^{10} + x * 2^6 - x$
 $= x * 32768 + x * 4096 + x * 2048 + x * 1024 + x * 64 - x = x * 39999$

7. $x \ll 16 - x \ll 9 - x \ll 4 - x \ll 2 - x \ll 1 - x$
 $= x * 2^{16} - x * 2^9 - x * 2^4 - x * 2^2 - x * 2^1 - x$
 $= x * 65536 - x * 512 - x * 16 - x * 4 - x * 2 - x = x * 64997$

8. $x \gg 4 = x / 2^4 = x / 16$

9. $x \gg 8 = x / 2^8 = x / 256$

10. $x \gg 13 = x / 2^{13} = x / 8192$

11. $x \gg 16 = x / 2^{16} = x / 65536$

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 2 of Text Book

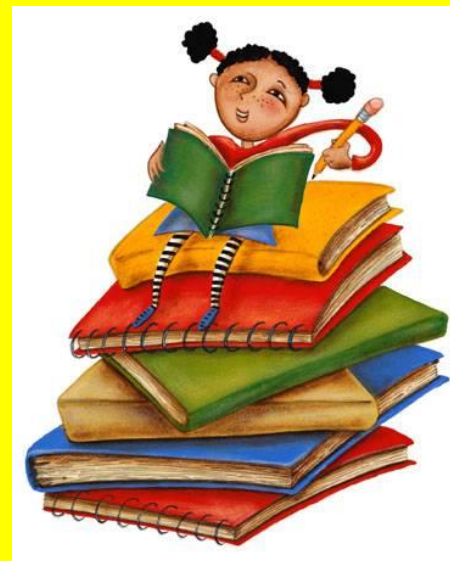
➤ Computer Systems : A Programmer's Perspective

- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**

TRY OUT ALL PRACTICE PROBLEMS

★ CHAPTER 2 - Practice Problems

- ★ 2.27(page 81 Solution page 142)
- ★ 2.28(page 82 Solution page 142)
- ★ 2.29(page 86 Solution page 143)
- ★ 2.30(page 86 Solution page 143)
- ★ 2.31(page 86 Solution page 143)
- ★ 2.32(page 87 Solution page 143)
- ★ 2.33(page 87 Solution page 144)
- ★ 2.34(page 90 Solution page 144)
- ★ 2.35(page 90 Solution page 144)
- ★ 2.36(page 91 Solution page 145)
- ★ 2.37(page 92 Solution page 145)
- ★ 2.38(page 93 Solution page 146)
- ★ 2.39(page 94 Solution page 146)
- ★ 2.40(page 94 Solution page 146)
- ★ 2.41(page 94 Solution page 146)
- ★ 2.42(page 97 Solution page 147)
- ★ 2.43(page 98 Solution page 147)
- ★ 2.44(page 99 Solution page 148)



SOC 2040

SYSTEM PROGRAMMING

END OF
WEEK 4 LECTURE 2

DATA REPRESENTATION

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

